CODE COMPREHENSION

Mei Nagappan



UNIVERSITY OF WATERLOO | FACULTY OF MATHEMATICS

GUIDELINE 4

Combine Search, Structural Navigation, and Examples;
Do Not Settle on One

Guideline 4: Combine Search, Structural Navigation, and Examples

- Use at least three navigation strategies in alternation: textual search, structural navigation, and example-based navigation.
- Textual: grep for identifiers. Structural: jump to definition, find references. Example-based: read tests, run in debugger.
- When one strategy stops giving new information, switch to another rather than searching harder with the same one.

Reasoning

- Successful developers consistently combined grep, jump-to-definition, and test reading
- Unsuccessful developers locked into one strategy (usually grep) and exhausted it
- Mixed-strategy navigation is the norm among professional developers
- Switching tools when stuck is a comprehension move, not just a tooling preference

Key Practices

- Strategy A (textual): grep for the user-visible string, error message, constant, or rare identifier
- Strategy B (structural): use jump-to-definition and find-references to walk the call graph
- Strategy C (example-based): read the tests of the module, then run one in a debugger
- After 5 minutes in any one strategy with no new information, switch to the next
- Keep a running note of what each strategy revealed; they should produce different facts

GUIDELINE 5

Run the Code; Static Reading Alone Is Misleading

Guideline 5: Run the Code; Static Reading Alone Is Misleading

- After forming a top-down hypothesis, confirm it by running the relevant code path.
- Set a breakpoint, paste the function into a REPL with a small input, or add a one-line print and run a test.
- Treat any surprise from the run as evidence that your reading was wrong, not a fluke.

Reasoning

- Effective developers regularly mixed dynamic investigation (debuggers, instrumentation)
- Runtime questions ('What is the value of X here?') were among the most frequent in fact-finding
- Developers reach for dynamic methods exactly when static reading produces unstable hypotheses
- Static reading misses dynamic dispatch, runtime configuration, monkey-patching, and concurrency interleavings

Key Practices

- After reading, write down what you expect a specific call to return or do
- Run that call; if you cannot run the whole app, paste the function into a REPL or write a 5-line test
- Compare actual to expected; if they differ, the reading was wrong; do not move on
- For methods, check the runtime type of self or the first argument; wrong type means wrong override
- For long pipelines, set a breakpoint at the uncertain boundary and inspect the actual value

GUIDELINE 6

Look for Why, Not Just What: Mine Comments, Commits, and Issue Links

Guideline 6: Look for Why: Mine Comments, Commits, and Issue Links

- The code shows you what is being done; the commit history, linked issues, and rationale-laden comments tell you why.
- Before changing code that looks weird, redundant, or 'easy to clean up', invest a few minutes finding the rationale.
- If you cannot find a rationale, treat that as a known knowledge gap and document your assumption.

Reasoning

- Rationale questions ('Why was it done this way?') are the most frequent and hardest to satisfy
- Missing rationale is the most painful gap newcomers faced when joining unfamiliar projects
- 'I refactored this because it looked redundant' is the classic way to reintroduce a fixed bug

Key Practices

- Read the comments around the suspect lines, not just within; the 'why' is usually one screen above
- Run git blame; look at the commit message of the line you are about to change
- Follow any issue or PR number in the commit message; read the issue and the linked PR discussion
- If the rationale is missing, write an ASSUMPTION note and consider asking the original author
- Treat comments that say 'DO NOT', 'MUST', or reference an issue number as hard constraints, not decoration

References (1/2)

- [1] Brooks, R. (1983). Towards a theory of the comprehension of computer programs. Int'l Journal of Man-Machine Studies, 18(6), 543-554.
- [2] Pennington, N. (1987). Stimulus structures and mental representations in expert comprehension. Cognitive Psychology, 19(3), 295-341.
- [3] Von Mayrhauser, A. & Vans, A. M. (1995). Program Comprehension During Software Maintenance. Computer, 28(8), 44-55.
- [4] Sajaniemi, J. (2002). An empirical analysis of roles of variables in novice-level programs. IEEE HCC '02, 37-39.
- [5] Robillard, M. P., Coelho, W. & Murphy, G. C. (2004). How effective developers investigate source code. IEEE TSE, 30(12), 889-903.
- [6] Sajaniemi, J. & Kuittinen, M. (2005). An experiment on using roles of variables in teaching. Computer Science Education, 15(1), 59-82.

References (2/2)

- [7] Sillito, J., Murphy, G. C. & De Volder, K. (2006). Questions programmers ask during software evolution tasks. FSE '06, 23-34.
- [8] LaToza, T. D., Garlan, D., Herbsleb, J. D. & Myers, B. A. (2007). Program comprehension as fact finding. ESEC/FSE 2007, 361-370.
- [9] Ko, A. J., DeLine, R. & Venolia, G. (2007). Information Needs in Collocated Software Development Teams. ICSE '07, 344-353.
- [10] LaToza, T. D. & Myers, B. A. (2010). Hard-to-answer questions about code. PLATEAU '10, Article 8.
- [11] Dagenais, B. et al. (2010). Moving into a new software project landscape. ICSE '10, 275-284.
- [12] Maalej, W., Tiarks, R., Roehm, T. & Koschke, R. (2014). On the Comprehension of Program Comprehension. TOSEM, 23(4), Article 31.